

Représentation numérique de l'information

ENIADB - 2023

TD 1

I - Les systèmes de numération

Définition :

.....

.....

.....

Expérience : On a tous eu un jour l'occasion de compter une quantité importante de petits objets : des pièces de monnaie, des billes, des cartes, etc. Notre compte fini, on en effectue un deuxième afin d'être certain de ne pas s'être trompé. Mais il est rare, malheureusement, de tomber deux fois sur le même résultat. Et là, notre esprit ingénieux nous conseille d'utiliser d'un stratagème pour ne pas se faire posséder une nouvelle fois par le grand nombre : on fait des petits paquets de 10 ! Et si cela ne suffit pas : avec 10 petits paquets de 10, nous formons un gros paquet de 100.

Nous réinventons un système de numération de **base 10**. Pourquoi « de base 10 », car pour obtenir un petit paquet, il faut 10 unités et pour obtenir un gros paquet, il faut 10 petits paquets. C'est notre système de numération actuel, composé de 10 symboles [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Pour passer au rang des dizaines [petits paquets], il faut 10 unités et pour passer au rang des centaines [gros paquets] il faut 10 dizaines.

I - 1 - Le décimal

Ces symboles sont :

En décimal, chaque symbole est appelé un chiffre, et un ensemble de chiffre est appelé un nombre. Dans le nombre, chaque chiffre a un rang : on parle d'unités, dizaines, centaines, milliers, etc ... On dit alors que le système décimal est un système Exemple de nombre décimal :

Rang →						
Poids →						
Chiffre →						
Valeur →						
Total →						

En une seule ligne on peut écrire que :

Remarques :

- *
- *
- *

I - 2 - Le binaire naturel

.....

.....

Ces symboles sont : En binaire naturel, chaque symbole est appelé un bit, et un ensemble de bit est appelé un mot. Un mot de 4 bits est appelé un Un mot de 8 bits est appelé un

Le binaire naturel est un système pondéré : chaque bit a un poids en fonction de sa position dans le mot. Exemple de nombre binaire :

Rang →						
Poids →						
Chiffre →						
Valeur →						
Total →						

Le nombre binaire s'écrit donc en décimal, ce qui s'écrit :

.....

Le symbole $\equiv$ [qui n'est pas le symbole « égale à »] signifie et se lit « correspond à ». Lien entre un nombre binaire et son équivalent en décimal :

.....

Pour interpréter un nombre en binaire naturel, il faut connaître les puissances de 2 :

Puissance de 2 →	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Valeur en décimal →													

Remarques :

- *
- *
- *
- *
- *

Le lien entre le poids d'un chiffre, son rang, et la base du système de numération est donné ci-contre. Ce lien est valable pour tous les systèmes de numération pondérés :



I - 3 - L'hexadécimal

.....

.....

Ces symboles sont :

Lien entre les 16 symboles hexadécimaux et leur équivalent en décimal :

Symbole hexadécimal →	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Equivalent en décimal →																

L'hexadécimal est un système pondéré :

Pour interpréter un nombre hexadécimal, il faut connaître les puissances de 16 :

Puissance de 16 →	16^5	16^4	16^3	16^2	16^1	16^0
Valeur en décimal →						

Exemple de nombre hexadécimal :

Rang →						
Poids →						
Chiffre →						
Valeur →						
Total →						

Lien entre un nombre hexadécimal et son équivalent en décimal :

.....

Remarques :

*

*

*

II - Conversion d'un nombre d'une base vers une autre

II - 1 - La conversion binaire → décimal

Principe :

.....

Exemples :

II - 2 - La conversion décimal → binaire

Principe :

.....

Exemples :

II - 3 - La conversion binaire → hexadécimal

Principe :

.....

Exemples :

II - 4 - La conversion hexadécimal → binaire

Principe :

.....

Exemples :

III - Opération sur les nombres binaires

III - 1 - Les opérations logiques

III - 1 - 1 - Le ET bits à bits

Cette opération, appliqué sur 2 nombres binaires, consiste à effectuer un ET logique entre deux bits du même rang. Exemples :

Le ET bits à bits permet de forcer à 0 certains bits d'un nombre binaire. Par exemple, pour forcer à 0 les bits de rang 1 et 6 du nombre 1101001011₍₂₎ il faut effectuer l'opération ci-contre :

ET	1	1	0	1	0	0	1	0	1	1
=										

III - 1 - 2 - Le OU bits à bits

Cette opération, appliqué sur 2 nombres binaires, consiste à effectuer un OU logique entre deux bits du même rang. Exemples :

Le OU bits à bits permet de forcer à 1 certains bits d'un nombre binaire. Par exemple, pour forcer à 1 les bits de rang 2 et 7 du nombre 1101001011₍₂₎ il faut effectuer l'opération ci-contre :

OU	1	1	0	1	0	0	1	0	1	1
=										

III - 1 - 3 - Le OU-Exclusif bits à bits

Cette opération, appliqué sur 2 nombres binaires, consiste à effectuer un OU-Exclusif logique entre deux bits du même rang. Exemples :

Le OU-Exclusif bits à bits permet de complémenter certains bits d'un nombre binaire. Par exemple, pour complémenter les bits de rang 0 et 5 du nombre $1101001011_{(2)}$ il faut effectuer l'opération ci-contre :

$$\begin{array}{r} 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 1 \\ \oplus \\ \hline \end{array}$$

III - 2 - Les opérations arithmétiques

III - 2 - 1 - L'addition de deux nombres binaires

Comme en décimal, l'addition de deux **nombres** binaires s'appuie sur l'utilisation d'une table d'addition, indiquant toutes les possibilités pour additionner deux **chiffres** binaires [deux bits]. Cette table d'addition élémentaire est donnée ci-contre : elle indique la somme **S** et la retenue éventuelle **R** lorsque l'on additionne deux bits **A** et **B**. On peut remarquer que les équations logiques de **S** et de **R** en fonction de **A** et **B** sont :

A	B	S	R
0	0		
0	1		
1	0		
1	1		

S =

R =

Le montage ci-contre, appelé « *demi-additionneur* », permet d'additionner 2 bits **A** et **B**, et donne en sortie la somme **S** ainsi qu'une retenue éventuelle **R** :

Exemples d'addition de 2 nombres binaires :

III - 2 - 2 - Le décalage à gauche

Cette opération consiste à décaler chaque bit d'un nombre binaire d'un rang vers la gauche : le bit de rang n est placé au rang $n+1$ et le LSB prend la valeur 0. Exemples :

.....

Remarque : décaler un nombre binaire d'un bit vers la gauche revient à multiplier ce nombre par 2.

III - 2 - 3 - Le décalage à droite

Cette opération consiste à décaler chaque bit d'un nombre binaire d'un rang vers la droite : le bit de rang n est placé au rang $n-1$ et le MSB prend la valeur 0. Exemples :

.....

Remarque : décaler un nombre binaire pair d'un bit vers la droite revient à diviser ce nombre par 2.

IV – Exercices d'application

IV – 1 – Convertissez en décimal les nombres binaires suivants :

A = 101011 ₍₂₎	E = 01110 ₍₂₎	I = 10111101 ₍₂₎	M = 100000 ₍₂₎	Q = 1110000 ₍₂₎	U = 10110000 ₍₂₎
B = 1111 ₍₂₎	F = 11011 ₍₂₎	J = 1010101 ₍₂₎	N = 101001 ₍₂₎	R = 10111011 ₍₂₎	V = 11011101 ₍₂₎
C = 10001 ₍₂₎	G = 1000001 ₍₂₎	K = 1101100 ₍₂₎	O = 1111010 ₍₂₎	S = 10000000 ₍₂₎	W = 10000111 ₍₂₎
D = 101000 ₍₂₎	H = 1111111 ₍₂₎	L = 001100 ₍₂₎	P = 1001000 ₍₂₎	T = 11110111 ₍₂₎	X = 10001000 ₍₂₎

IV – 2 – Convertissez en binaire naturel les nombres décimaux suivants :

A = 16 ₍₁₀₎	E = 63 ₍₁₀₎	I = 256 ₍₁₀₎	M = 457 ₍₁₀₎	Q = 1473 ₍₁₀₎	U = 5000 ₍₁₀₎
B = 64 ₍₁₀₎	F = 65 ₍₁₀₎	J = 255 ₍₁₀₎	N = 579 ₍₁₀₎	R = 2004 ₍₁₀₎	V = 6237 ₍₁₀₎
C = 33 ₍₁₀₎	G = 100 ₍₁₀₎	K = 257 ₍₁₀₎	O = 993 ₍₁₀₎	S = 3674 ₍₁₀₎	W = 7088 ₍₁₀₎
D = 130 ₍₁₀₎	H = 41 ₍₁₀₎	L = 312 ₍₁₀₎	P = 1016 ₍₁₀₎	T = 4910 ₍₁₀₎	X = 8653 ₍₁₀₎

IV – 3 – Convertissez en binaire naturel les nombres hexadécimaux suivants :

A = 1234 ₍₁₆₎	E = C5D9 ₍₁₆₎	I = FFFFF ₍₁₆₎	M = 553E32 ₍₁₆₎	Q = 5D3F4B6 ₍₁₆₎	U = 422E7498 ₍₁₆₎
B = ABCD ₍₁₆₎	F = 3E7B4 ₍₁₆₎	J = 101010 ₍₁₆₎	N = 123456 ₍₁₆₎	R = 9A40C27 ₍₁₆₎	V = BE4A3EDF ₍₁₆₎
C = 47B9 ₍₁₆₎	G = A850B ₍₁₆₎	K = 842814 ₍₁₆₎	O = CCCDDDD ₍₁₆₎	S = B3EC8AA ₍₁₆₎	W = C3EDAFCB ₍₁₆₎
D = F021 ₍₁₆₎	H = 76534 ₍₁₆₎	L = BCDFEA ₍₁₆₎	P = 720A96 ₍₁₆₎	T = 10F01F1 ₍₁₆₎	X = 420A6D79 ₍₁₆₎

IV – 4 – Convertissez en hexadécimal les nombres binaires suivants :

A = 1011 ₍₂₎	E = 10010 ₍₂₎	I = 110010 ₍₂₎	M = 1011011 ₍₂₎	Q = 10110111 ₍₂₎	U = 111111111 ₍₂₎
B = 1100 ₍₂₎	F = 10111 ₍₂₎	J = 100000 ₍₂₎	N = 1101011 ₍₂₎	R = 10100011 ₍₂₎	V = 101111110 ₍₂₎
C = 1111 ₍₂₎	G = 10100 ₍₂₎	K = 101010 ₍₂₎	O = 1001011 ₍₂₎	S = 11111000 ₍₂₎	W = 101101011 ₍₂₎
D = 1001 ₍₂₎	H = 11110 ₍₂₎	L = 111101 ₍₂₎	P = 1100010 ₍₂₎	T = 11011110 ₍₂₎	X = 100010000 ₍₂₎

IV – 5 – Convertissez en décimal les nombres hexadécimaux suivants :

A = E3 ₍₁₆₎	E = 12D ₍₁₆₎	I = 1010 ₍₁₆₎	M = 6C82 ₍₁₆₎	Q = 462A7 ₍₁₆₎	U = 523014 ₍₁₆₎
B = 2C ₍₁₆₎	F = 743 ₍₁₆₎	J = FEDC ₍₁₆₎	N = 430B ₍₁₆₎	R = 963BE ₍₁₆₎	V = 9E46FA ₍₁₆₎
C = F4 ₍₁₆₎	G = A1B ₍₁₆₎	K = F000 ₍₁₆₎	O = 9A10 ₍₁₆₎	S = FBF71 ₍₁₆₎	W = 20C736 ₍₁₆₎
D = 87 ₍₁₆₎	H = 506 ₍₁₆₎	L = 7451 ₍₁₆₎	P = C7C6 ₍₁₆₎	T = A057C ₍₁₆₎	X = F01F28 ₍₁₆₎

IV – 6 – Convertissez en hexadécimal les nombres décimaux suivants :

A = 12 ₍₁₀₎	E = 157 ₍₁₀₎	I = 1789 ₍₁₀₎	M = 3100 ₍₁₀₎	Q = 4096 ₍₁₀₎	U = 1111 ₍₁₀₎
B = 20 ₍₁₀₎	F = 547 ₍₁₀₎	J = 1515 ₍₁₀₎	N = 5781 ₍₁₀₎	R = 2048 ₍₁₀₎	V = 2222 ₍₁₀₎
C = 35 ₍₁₀₎	G = 888 ₍₁₀₎	K = 1918 ₍₁₀₎	O = 6951 ₍₁₀₎	S = 5000 ₍₁₀₎	W = 3333 ₍₁₀₎
D = 99 ₍₁₀₎	H = 470 ₍₁₀₎	L = 2000 ₍₁₀₎	P = 7810 ₍₁₀₎	T = 2050 ₍₁₀₎	X = 4444 ₍₁₀₎

IV – 7 – Que deviennent chacun des nombres hexadécimaux suivants après avoir subi un décalage à gauche de 3 bits ? Vous exprimerez vos réponses dans le système de numération hexadécimal :

A = 45 ₍₁₆₎	E = 179 ₍₁₆₎	I = 1212 ₍₁₆₎	M = 76851 ₍₁₆₎	Q = 10000 ₍₁₆₎	U = 365DDE ₍₁₆₎
B = B9 ₍₁₆₎	F = 3EE ₍₁₆₎	J = 744B ₍₁₆₎	N = CBB94 ₍₁₆₎	R = CCDCD ₍₁₆₎	V = C9D9E9 ₍₁₆₎
C = 3C ₍₁₆₎	G = 9C2 ₍₁₆₎	K = 95E9 ₍₁₆₎	O = 30210 ₍₁₆₎	S = 65456 ₍₁₆₎	W = ACB540 ₍₁₆₎
D = D0 ₍₁₆₎	H = 46A ₍₁₆₎	L = 6000 ₍₁₆₎	P = 852AD ₍₁₆₎	T = 98789 ₍₁₆₎	X = 70E45C ₍₁₆₎

IV – 8 – Que deviennent chacun des nombres décimaux suivants après avoir subi un décalage à droite de 2 bits ? Vous exprimerez vos réponses dans le système de numération décimal :

A = 25 ₍₁₀₎	E = 113 ₍₁₀₎	I = 347 ₍₁₀₎	M = 993 ₍₁₀₎	Q = 1763 ₍₁₀₎	U = 777 ₍₁₀₎
B = 16 ₍₁₀₎	F = 128 ₍₁₀₎	J = 555 ₍₁₀₎	N = 1024 ₍₁₀₎	R = 2150 ₍₁₀₎	V = 512 ₍₁₀₎
C = 7 ₍₁₀₎	G = 189 ₍₁₀₎	K = 680 ₍₁₀₎	O = 1025 ₍₁₀₎	S = 3988 ₍₁₀₎	W = 1020 ₍₁₀₎
D = 39 ₍₁₀₎	H = 214 ₍₁₀₎	L = 871 ₍₁₀₎	P = 1026 ₍₁₀₎	T = 4096 ₍₁₀₎	X = 2049 ₍₁₀₎

IV – 9 – Effectuez en binaire les additions suivantes, puis convertissez le résultat en décimal :

A = 10111010 ₍₂₎ + 1101100 ₍₂₎	G = 11111111 ₍₂₎ + 1 ₍₂₎	M = 11111111 ₍₂₎ + 10110011 ₍₂₎
B = 1010100 ₍₂₎ + 11111111 ₍₂₎	H = 1101111011 ₍₂₎ + 10110111 ₍₂₎	N = 1001001110 ₍₂₎ + 110111101 ₍₂₎
C = 11111 ₍₂₎ + 11111 ₍₂₎	I = 10011011 ₍₂₎ + 11111111 ₍₂₎	O = 111011101 ₍₂₎ + 1111110101 ₍₂₎
D = 10010010 ₍₂₎ + 1101100 ₍₂₎	J = 10101010 ₍₂₎ + 111000111 ₍₂₎	P = 1001110101 ₍₂₎ + 1111000111 ₍₂₎
E = 1000 ₍₂₎ + 1001 ₍₂₎	K = 11010000 ₍₂₎ + 10101111 ₍₂₎	Q = 1010101010 ₍₂₎ + 1111111011 ₍₂₎
F = 1110111 ₍₂₎ + 1011111 ₍₂₎	L = 11111111 ₍₂₎ + 11111101 ₍₂₎	R = 1101111011 ₍₂₎ + 1011110111 ₍₂₎